

사용자 문제를 데이터로 정의하고, 백엔드와 AI 모델로 해결하는 소프트웨어 엔지니어 이찬호입니다



(+82) 010-2365-1837



ibear6954@gmail.com



github.com/teddygood



teddygood.github.io



linkedin.com/in/teddygood

About me



이찬호

CHANHO LEE

✉ ibear6954@gmail.com

☎ (+82) 010-2365-1837

GitHub Blog LinkedIn

Skills

- 언어: Python, SQL, C/C++, Java, JavaScript, TypeScript, CUDA, Rust
- 프론트엔드: React, Docusaurus
- 백엔드: Django, Flask, FastAPI, Node.js, Spring Boot, MySQL, MongoDB, PostgreSQL
- 클라우드/인프라: AWS, GCP, Docker, Kubernetes, Terraform, GitHub Actions
- 데이터/AI: Spark, Airflow, PyTorch, TensorFlow, LangChain, LangGraph, vLLM

Education

광운대학교 컴퓨터공학과 졸업

2018.03 - 2025.08

주요 수강 과목: 자료구조, 알고리즘, 데이터베이스, 운영체제, 컴퓨터구조, 컴퓨터네트워크, 시스템프로그래밍, 객체지향프로그래밍, 머신러닝, 확률과통계, 선형대수학, GPU 프로그래밍

Work Experience

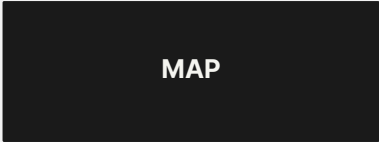
AWS 클라우드 서포트 엔지니어 인턴

2024.12 - 2025.06

Strength

계속 배우는 성장
함께 성장하는 소통
검증하는 해결

빠르게 변하는 기술 속에서도 배움을 멈추지 않고, 공부한 내용을 기록하고 나누며 한 단계씩 성장하려 합니다.
혼자 완벽한 답을 찾기보다 진행 상황과 고민을 투명하게 공유하고, 피드백을 반영해 더 나은 방향을 만듭니다.
문제를 회피하지 않고 먼저 정의한 뒤, 가설을 세우고 검증하며 실행 가능한 해결책으로 끝까지 연결합니다.



문제를 운영 가능한 구조와 검증 가능한 증거로 바꿉니다.

이 책은 기능 목록보다 문제 인식, 담당 역할, 해결 과정, 검증 증거를 먼저 보여주도록 구성했습니다.

PROJECT	PROBLEM	STACK	EVIDENCE
추천 백엔드	이벤트와 지표 관측이 분리되어 신뢰와 추적이 어려웠습니다.	Go, Kafka, PostgreSQL, Elasticsearch, Grafana	API test, Docker smoke
Optiver	경매 신호가 급변해 60초 뒤 가격 예측이 흔들렸습니다.	LightGBM, feature engineering, time-series CV	CV MAE 5.8259
Pyodide SIMD	SIMD 빌드가 패키지와 CI에서 검증되어야 했습니다.	WebAssembly, C, packaging, OpenBLAS	pyodide PR #5880 merged
LLM PR Reviewer	multi-file diff와 LLM 응답을 안정적으로 처리해야 했습니다.	Bedrock, Lambda, DynamoDB, OpenSearch, API Gateway	100+ test PRs

추천 백엔드

개인 프로젝트 / Go, Kafka, PostgreSQL, Elasticsearch, Grafana

상황	추천 서비스는 사용자 이벤트, 실험 배정, 지표 집계, 장애 관측이 분리되면 결과를 신뢰하기 어렵고 디버깅도 늦어집니다.
과제	추천 모델 이전에 이벤트를 안정적으로 저장하고, 실험 노출과 지표를 재현 가능하게 만들며, 장애 대응 시간을 줄이는 운영 구조가 필요했습니다.
행동	Kafka event pipeline, idempotent projection, deterministic assignment, SRM warning, Prometheus와 Grafana 모니터링을 나누어 구현했습니다.
결과	API 테스트와 Docker smoke로 주요 경로를 검증했고, 추천 백엔드의 데이터 흐름과 운영 지표를 포트폴리오 단위로 정리했습니다.
배운 점	추천 품질 이전에 이벤트 정의, 실험 기준, 관측 가능성이 맞아야 제품 판단이 가능하다는 점을 얻었습니다.

추천 백엔드

개인 프로젝트 / Go, Kafka, PostgreSQL, Elasticsearch, Grafana

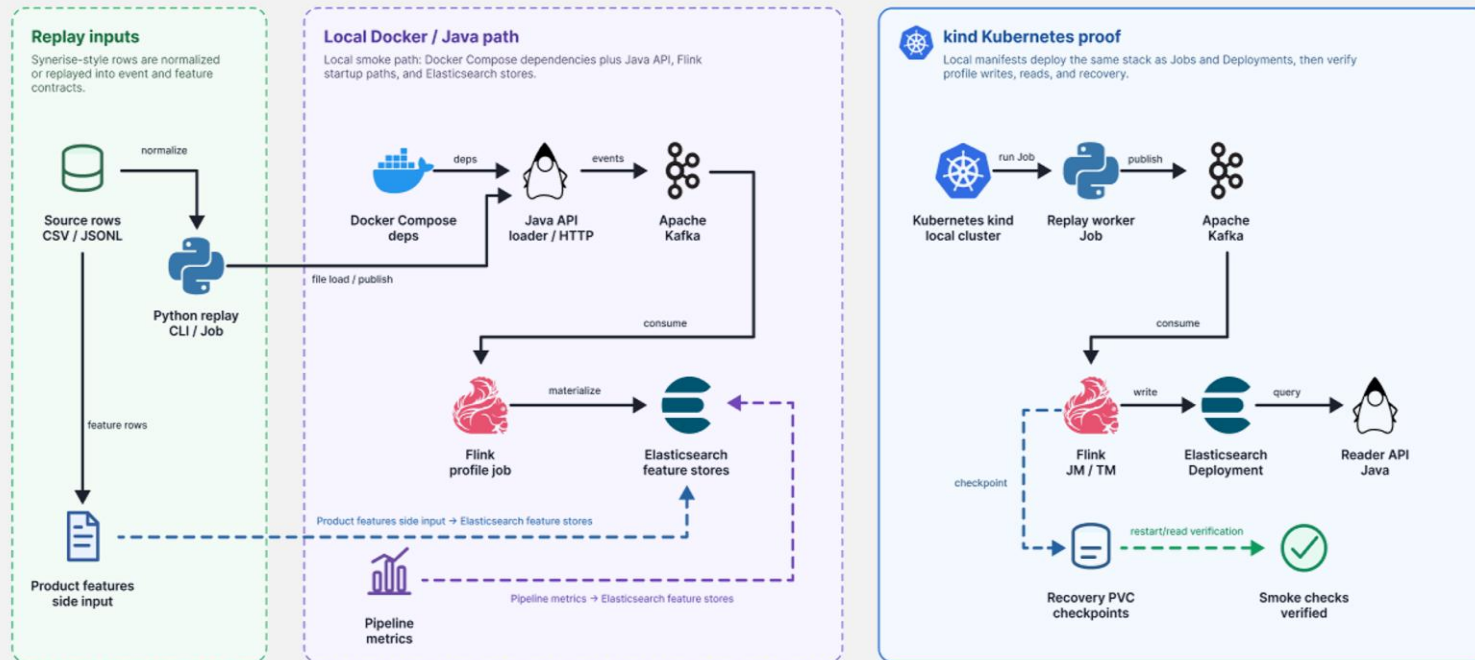
ROLE + ARCH 01

프로젝트 개요

- replay input -> Kafka/Flink -> Elasticsearch로 이어지는 흐름을 분리해, 추천 feature가 어디서 생성되고 저장되는지 추적 가능하게 설계했습니다.
- Docker 로컬 smoke path와 kind Kubernetes proof를 함께 두어, 같은 이벤트 계약과 복구 흐름이 유지되는지 검증했습니다.

내 역할

- Synerise 스타일 원천 row를 event/feature contract로 정규화하고, Java API loader와 Python replay job으로 재현 가능한 입력 경로를 만들었습니다.
- Flink profile job, Elasticsearch feature store, checkpoint 기반 recovery를 묶어 관측 지표와 smoke check로 운영 가능성을 확인했습니다.



Optiver Trading at the Close

Kaggle 대회 / LightGBM, feature engineering, time-series CV

상황	장 마감 경매 데이터에서는 체결, 호가, 불균형 신호가 빠르게 바뀌며 60초 뒤 가격 변화를 안정적으로 예측해야 했습니다.
과제	시간 누수를 피하면서 auction과 orderbook feature를 만들고, 스트리밍 추론 환경에서도 같은 feature를 유지해야 했습니다.
행동	124개 feature, Numba triplet imbalance, date_id 5-fold와 5-day purge, LightGBM ensemble, streaming API cache를 구성했습니다.
결과	private leaderboard 981 / 3,225 teams, score 5.4744를 기록했고, 최종 CV MAE는 5.8259였습니다.
배운 점	대회 성능은 feature 수보다 검증 설계와 온라인 추론 제약을 얼마나 일관되게 맞추는지가 중요하다는 점을 얻었습니다.

Optiver Trading at the Close

Kaggle 대회 / LightGBM, feature engineering, time-series CV

ROLE + ARCH 02

프로젝트 개요

- 장 마감 auction/orderbook 데이터에서 누수 없이 60초 뒤 가격 변화를 예측해야 했고, 체결/호가/imbalance 신호의 시간 경계를 분리했습니다.
- offline CV와 제출 파이프라인의 feature 정의가 달라지지 않도록, 학습과 추론에서 같은 생성 규칙을 유지했습니다.

내 역할

- EDA로 target, near_price, far_price, imbalance_size의 시간별 분포와 결측/이상 패턴을 확인하고 124개 feature를 설계했습니다.
- date_id 5-fold와 5-day purge로 시간 누수를 제어하고, LightGBM ensemble과 streaming API cache로 제출 흐름까지 검증했습니다.

아키텍처 이미지 자리

EDA / Feature / CV / Inference

Pyodide WebAssembly SIMD

오픈소스 기여 / WebAssembly, C, Python packaging, OpenBLAS

상황	Pyodide에는 WASM SIMD 지원과 OpenBLAS SIMD 빌드가 실제 패키지와 CI에서 어떻게 검증되는지 확인할 기준이 부족했습니다.
과제	SIMD intrinsics가 동작하는지, OpenBLAS SIMD를 기본값으로 둘 만큼 유효한지 재현 가능한 방식으로 판단해야 했습니다.
행동	test-simd 패키지, scalar 비교 테스트, -msimd128 OpenBLAS 빌드, warm-up을 반영한 벤치마크, scipy 테스트를 정리했습니다.
결과	SIMD 테스트 패키지와 OpenBLAS SIMD 기본 빌드 관련 PR이 병합됐고, pyodide-build cleanup CLI 후속 PR도 병합됐습니다.
배운 점	성능 개선은 빠른 수치보다 측정 조건, 워밍업, 기본값 변경의 유지보수 비용을 함께 설득해야 한다는 점을 얻었습니다.

Pyodide WebAssembly SIMD

오픈소스 기여 / WebAssembly, C, Python packaging, OpenBLAS

ROLE + ARCH 03

프로젝트 개요

- Pyodide의 WASM SIMD 지원은 빌드 성공보다, SIMD intrinsic이 scalar 결과와 같고 OpenBLAS 기본 빌드 변경이 안전한지 증명해야 했습니다.
- 패키지 테스트, benchmark recipe, CI 로그가 함께 남아야 리뷰어가 성능 개선과 호환성 위험을 분리해 판단할 수 있었습니다.

내 역할

- test-simd 패키지에 SIMD/scalar 결과 비교를 추가하고, warm-up과 측정 조건을 분리한 benchmark recipe를 정리했습니다.
- macOS ARM, GCC flag, OpenBLAS 기본값 변경 등 리뷰 이슈를 재현해 수정하고, pyodide-build 후속 PR까지 연결했습니다.

아키텍처 이미지 자리

SIMD Test / OpenBLAS / CI / Cleanup

LLM-based PR Reviewer

AWS 인턴십 팀 프로젝트 / Bedrock, Lambda, DynamoDB, OpenSearch, API Gateway

상황	워크숍 참가자가 PR을 만들면 AI가 리뷰를 남기는 흐름이 필요했지만, PR diff는 여러 파일로 나뉘고 LLM 응답 품질도 쉽게 흔들렸습니다.
과제	PR 생성부터 diff 수집, 리뷰 가이드라인 검색, LLM 코멘트 생성, GitHub 반영까지 안정적으로 이어지는 서버리스 파이프라인이 필요했습니다.
행동	GitHub Webhook, API Gateway, Lambda, DynamoDB, S3, OpenSearch, Bedrock, GitHub API를 연결하고 multi-file diff 통합과 prompt 개선 루프를 설계했습니다.
결과	100개 이상의 테스트 PR로 품질을 점검했고, 45명 워크숍 참가자가 실습을 완주했으며 CSAT 4.92/5를 기록했습니다.
배운 점	LLM 시스템은 모델 호출보다 입력 구조화, 검색 기준, 평가 루프, 운영 알림이 함께 맞아야 서비스처럼 동작한다는 점을 얻었습니다.

LLM-based PR Reviewer

AWS 인턴십 팀 프로젝트 / Bedrock, Lambda, DynamoDB, OpenSearch, API Gateway

ROLE + ARCH 04

프로젝트 개요

- 참가자가 PR을 만들면 AI가 리뷰 코멘트를 남기는 흐름을 목표로, GitHub webhook부터 모델 호출과 코멘트 반영까지 서버리스 파이프라인으로 설계했습니다.
- multi-file diff, 긴 PR 문맥, LLM 응답 품질 편차 때문에 diff 구조화, 검색 기준, 실패 알림을 함께 설계해야 했습니다.

내 역할

- Unified Diff를 파일 단위로 전처리하고 OpenSearch 검색 문맥과 Bedrock 프롬프트를 조합해 리뷰 입력을 안정화했습니다.
- DynamoDB Streams, Lambda, SNS로 비동기 처리와 실패 알림을 구성하고, 100개 이상의 test PR로 코멘트 품질을 점검했습니다.

아키텍처 이미지 자리

GitHub / Lambda / DynamoDB / Bedrock / GitHub API

CLOSING MESSAGE

END

저는 문제를 구조화하고
검증 가능한 결과로 남기는 엔지니어입니다.

이 포트폴리오는 프로젝트별 기능 목록이 아니라, 문제를 어떻게 인식했고 어떤 역할로 해결했으며
무엇으로 검증했는지를 보여주기 위한 구성입니다.

STAR
역할 + 아키텍처
검증 증거